

Name: D. Sri Anjaneya
Register Number: 192425219

CO5 – AT2
MLA 0201 – Fundamentals of Machine Learning

Annexure A

Pseudocode Writing Task (Algorithm Design)

Question 1: Monte Carlo Sampling for Probability Estimation

A logistics company wants to estimate the probability of on-time deliveries under uncertain traffic conditions using **Monte Carlo Sampling**.

Task:

Write a pseudocode algorithm to:

- Generate random samples representing delivery conditions.
- Simulate multiple delivery trials.
- Estimate the probability of on-time delivery.
- Display the estimated probability after all simulations.

Question 2: ϵ -Greedy Algorithm for K-Armed Bandit

An online streaming platform recommends one of several movies to maximize user engagement. The platform must balance **exploration** and **exploitation**.

Task:

Write a pseudocode algorithm to:

- Initialize rewards for all movie recommendations.
- Implement the **ϵ -greedy exploration strategy**.
- Update reward estimates after each user interaction.
- Identify the movie with the highest expected reward.

Question 3: Value Iteration for Robot Path Planning

A warehouse robot must determine the optimal path to reach a destination while maximizing rewards and avoiding obstacles.

Task:

Write a pseudocode algorithm to:

- Initialize the value function for all states.
- Update state values using the **Bellman optimality equation**.
- Repeat the process until convergence.

- Output the optimal value function and policy.

Question 4: Temporal Difference (TD) Learning for Game Score Prediction

A game AI agent learns the value of each game state based on rewards received after every move.

Task:

Write a pseudocode algorithm to:

- Initialize state-value estimates.
- Observe the current state, reward, and next state.
- Update the state value using the **Temporal Difference (TD) learning rule**.
- Continue until all game episodes are completed.

Question 5: Policy Iteration for Autonomous Drone Navigation

An autonomous drone must learn the optimal navigation policy to deliver packages while minimizing energy consumption.

Task:

Write a pseudocode algorithm to:

- Initialize a random policy.
- Perform **policy evaluation** to compute state values.
- Improve the policy by selecting the best action for each state.
- Repeat until the policy becomes stable and output the optimal policy.

1. Monte Carlo Sampling for Probability Estimation

Objective: Estimate the probability of on-time delivery under uncertain traffic conditions by simulating many delivery trials.

Input:

- Number of simulations N
- Traffic conditions and their probabilities
- Delivery time limit

Output: Estimated probability of on-time delivery

Pseudocode:

```
ALGORITHM MonteCarloDeliveryProbability

INPUT:
    N ← Number of simulations
    OnTimeCount ← 0

FOR trial ← 1 TO N DO

    // Generate a random traffic condition
    Traffic ← RANDOM_CHOICE(LOW, MEDIUM, HIGH)

    // Simulate delivery time based on traffic
    IF Traffic = LOW THEN
        DeliveryTime ← RANDOM_VALUE(20, 30)
    ELSE IF Traffic = MEDIUM THEN
        DeliveryTime ← RANDOM_VALUE(30, 45)
    ELSE
        DeliveryTime ← RANDOM_VALUE(45, 60)
    END IF

    // Check whether delivery is on time
    IF DeliveryTime ≤ DeliveryTimeLimit THEN
        OnTimeCount ← OnTimeCount + 1
```

```
END IF

END FOR

// Estimate probability
EstimatedProbability ← OnTimeCount / N

DISPLAY "Estimated Probability of On-Time Delivery = ",
    EstimatedProbability

END ALGORITHM
```

Explanation:

The algorithm performs N independent delivery simulations. For every trial, a random traffic condition is generated and a corresponding delivery time is simulated. If the delivery is within the specified time limit, the successful trial is counted. Finally:

Estimated Probability = Number of On-Time Deliveries / Total Simulations

2. ϵ -Greedy Algorithm for K-Armed Bandit

Objective: Select movies that maximize user engagement while balancing **exploration** and **exploitation**.

Input:

- Number of movies K
- Number of interactions T
- Exploration probability ϵ

Output: Movie with the highest estimated reward

Pseudocode:

```
ALGORITHM EpsilonGreedyMovieRecommendation
```

```
INPUT:
```

```
     $K \leftarrow$  Number of movies
```

```
     $T \leftarrow$  Number of user interactions
```

```
     $\epsilon \leftarrow$  Exploration probability
```

```
FOR each movie  $i \leftarrow 1$  TO  $K$  DO
```

```
    RewardEstimate[ $i$ ]  $\leftarrow 0$ 
```

```
    RewardCount[ $i$ ]  $\leftarrow 0$ 
```

```
END FOR
```

```
FOR interaction  $\leftarrow 1$  TO  $T$  DO
```

```
    // Generate a random number
```

```
     $r \leftarrow \text{RANDOM}(0, 1)$ 
```

```
    // Exploration
```

```
    IF  $r < \epsilon$  THEN
```

```
        Movie  $\leftarrow \text{RANDOM\_CHOICE}(1, K)$ 
```

```
    // Exploitation
```

```
    ELSE
```

```

    Movie ← movie having maximum RewardEstimate
END IF

// Observe user's reward/engagement
Reward ← GET_USER_REWARD(Movie)

// Update number of times movie was selected
RewardCount[Movie] ← RewardCount[Movie] + 1

// Update estimated reward
RewardEstimate[Movie] ←
    RewardEstimate[Movie] +
    (Reward - RewardEstimate[Movie]) / RewardCount[Movie]

END FOR

// Find movie with highest expected reward
BestMovie ← ARG_MAX(RewardEstimate)

DISPLAY "Movie with Highest Expected Reward = ", BestMovie
DISPLAY "Expected Reward = ", RewardEstimate[BestMovie]

END ALGORITHM

```

Explanation:

With probability ϵ , the algorithm **explores** by selecting a random movie. With probability $1 - \epsilon$, it **exploits** the movie having the highest estimated reward. After every interaction, the estimated reward is updated using the observed user engagement.

3. Value Iteration for Robot Path Planning

Objective: Find the optimal path for a warehouse robot by maximizing the total expected reward while avoiding obstacles.

Input:

- Set of states S
- Set of actions A
- Transition probabilities
- Rewards
- Discount factor γ
- Convergence threshold θ

Output: Optimal value function and optimal policy

Pseudocode:

```
ALGORITHM ValueIteration
```

```
INPUT:
```

```
States  $S$ 
```

```
Actions  $A$ 
```

```
TransitionModel  $P$ 
```

```
Reward  $R$ 
```

```
DiscountFactor  $\gamma$ 
```

```
Threshold  $\theta$ 
```

```
// Initialize value function
```

```
FOR each state  $s$  IN  $S$  DO
```

```
    IF  $s$  is an obstacle THEN
```

```
         $V[s] \leftarrow -\infty$ 
```

```
    ELSE IF  $s$  is terminal THEN
```

```
         $V[s] \leftarrow R[s]$ 
```

```
    ELSE
```

```
         $V[s] \leftarrow 0$ 
```

```
    END IF
```

```
END FOR
```

```
REPEAT
```

```
     $\Delta \leftarrow 0$ 
```

```
    FOR each state  $s$  IN  $S$  DO
```

```
        IF  $s$  is terminal OR  $s$  is obstacle THEN
```

```
            CONTINUE
```

```
        END IF
```

```
        OldValue  $\leftarrow V[s]$ 
```

```
        // Bellman optimality update
```

```
         $V[s] \leftarrow \text{MAX over actions } a \text{ of}$ 
```

```
            [  $R(s,a) +$ 
```

```
               $\gamma \times \sum P(s'|s,a) \times V[s']$  ]
```

```
         $\Delta \leftarrow \text{MAX}(\Delta, \text{ABS}(\text{OldValue} - V[s]))$ 
```

```
    END FOR
```

```
UNTIL  $\Delta < \theta$ 
```

```
// Extract optimal policy
```

```
FOR each non-terminal state  $s$  IN  $S$  DO
```

```
    Policy[ $s$ ]  $\leftarrow$  action  $a$  that maximizes
```

```
        [  $R(s,a) +$ 
```

```
           $\gamma \times \sum P(s'|s,a) \times V[s']$  ]
```



```
END FOR
```

```
DISPLAY "Optimal Value Function:"
```

```
DISPLAY V
```

```
DISPLAY "Optimal Policy:"
```

```
DISPLAY Policy
```

```
END ALGORITHM
```

Explanation:

Value Iteration repeatedly applies the **Bellman optimality equation**. Each state's value is updated using the best possible action. The process stops when the maximum change in values becomes smaller than the threshold θ . The optimal policy is then obtained by selecting the action with the highest expected value.

4. Temporal Difference (TD) Learning for Game Score Prediction

Objective: Learn the value of game states by updating estimates after every move using observed rewards.

Input:

- Game episodes
- Learning rate α
- Discount factor γ

Output: Learned state-value estimates

Pseudocode:

```
ALGORITHM TD_Learning
```

```
INPUT:
```

```
    NumberOfEpisodes
```

```
    LearningRate  $\alpha$ 
```

```
    DiscountFactor  $\gamma$ 
```

```
// Initialize state values
```

```
FOR each state  $s$  DO
```

```
     $V[s] \leftarrow 0$ 
```

```
END FOR
```

```
FOR episode  $\leftarrow 1$  TO NumberOfEpisodes DO
```

```
    // Start a new game
```

```
    State  $\leftarrow$  INITIAL_STATE()
```

```
    WHILE State is not terminal DO
```

```
        // Observe next state and reward
```

```
        (NextState, Reward)  $\leftarrow$  TAKE_ACTION(State)
```

```
        // Calculate TD target
```

```
    TD_Target  $\leftarrow$  Reward +  $\gamma \times V[\text{NextState}]$ 

    // Calculate TD error
    TD_Error  $\leftarrow$  TD_Target - V[State]

    // Update current state value
    V[State]  $\leftarrow$  V[State] +  $\alpha \times$  TD_Error

    // Move to next state
    State  $\leftarrow$  NextState

END WHILE

END FOR

DISPLAY "Learned State Values:"
DISPLAY V

END ALGORITHM
```

Explanation:

TD Learning updates the value of a state **after every move**, rather than waiting until the entire game is finished. The update rule is:

$$V(S) \leftarrow V(S) + \alpha [R + \gamma V(S') - V(S)]$$

Here, α controls the learning rate and γ determines the importance of future rewards.

5. Policy Iteration for Autonomous Drone Navigation

Objective: Find the optimal navigation policy for a drone while maximizing rewards or minimizing energy consumption.

Input:

- States S
- Actions A
- Transition probabilities
- Rewards
- Discount factor γ

Output: Optimal policy

Pseudocode:

```
ALGORITHM PolicyIteration
```

```
INPUT:
```

```
    States  $S$ 
```

```
    Actions  $A$ 
```

```
    TransitionModel  $P$ 
```

```
    Reward  $R$ 
```

```
    DiscountFactor  $\gamma$ 
```

```
// Initialize a random policy
```

```
FOR each state  $s$  IN  $S$  DO
```

```
    Policy[ $s$ ]  $\leftarrow$  RANDOM_CHOICE(AvailableActions( $s$ ))
```

```
END FOR
```

```
REPEAT
```

```
    // -----
```

```
    // Policy Evaluation
```

```
    // -----
```

```
    FOR each state  $s$  IN  $S$  DO
```

```

     $V[s] \leftarrow 0$ 
END FOR

REPEAT

     $\Delta \leftarrow 0$ 

    FOR each state  $s$  IN  $S$  DO

        IF  $s$  is terminal THEN
            CONTINUE
        END IF

        OldValue  $\leftarrow V[s]$ 

         $a \leftarrow \text{Policy}[s]$ 

         $V[s] \leftarrow R(s,a) +$ 
             $\gamma \times \sum P(s'|s,a) \times V[s']$ 

         $\Delta \leftarrow \text{MAX}(\Delta, \text{ABS}(\text{OldValue} - V[s]))$ 

    END FOR

UNTIL  $\Delta < \theta$ 

// -----
// Policy Improvement
// -----

PolicyStable  $\leftarrow \text{TRUE}$ 

```

```

FOR each state s IN S DO

    IF s is terminal THEN
        CONTINUE
    END IF

    OldAction ← Policy[s]

    // Select best action
    Policy[s] ← action a that maximizes
        [ R(s,a) +
           $\gamma \times \sum P(s'|s,a) \times V[s']$  ]

    IF OldAction ≠ Policy[s] THEN
        PolicyStable ← FALSE
    END IF
END FOR
UNTIL PolicyStable = TRUE

DISPLAY "Optimal State Values:"
DISPLAY V
DISPLAY "Optimal Drone Navigation Policy:"
DISPLAY Policy
END ALGORITHM

```

Explanation:

Policy Iteration works in two major stages:

1. **Policy Evaluation:** Calculates the value of each state under the current policy.
2. **Policy Improvement:** Checks all possible actions and changes the policy to the action with the highest expected value.

These two steps continue until no policy changes occur. At that point, the policy is considered **optimal**.